Автономная некоммерческая организация высшего образования
«Санкт-Петербургский институт экономики и управления»



Кафедра **09.02.07 Информационные системы и программирование**

ПРАКТИЧЕСКАЯ РАБОТА № 8

Дисциплина(модуль)/предмет **МДК.03.02**

Тема: **Устранение уязвимостей API**

ВАРИАНТ № **3**

Студент (ка)/учащийся: **Бойцов Даниил Антонович** / _____ /
Ф. И. О. подпись

Группа / Класс **И13-24-11**

Руководитель: **Чудинова А.А.** /_____ /
Должность, степень, Фамилия и инициалы подпись

Дата представления работы «04» ноября 2026 г.

Работа аттестована с оценкой _____
прописью

Санкт-Петербург
2026 г.

Содержание

1 Уязвимый код и контракт защиты.....3
2 Безопасная реализация и инфраструктура.....4
3 Интеграционный тест закрытия уязвимости.....5
4 Реальный снимок проверки и ограничения.....6
Подтверждающие материалы 7
Заключение 8
Список использованных источников 9

1 Уязвимый код и контракт защиты

Цель работы: устранить подмену показаний Climate Control и подтвердить результат интеграционным тестом. Исходная учебная библиотека принимает значения без подписи, не связывает SensorId с подтверждённым устройством и ошибочно проверяет температуру. Она не публикуется как рабочий сетевой сервис.

```
// Уязвимая учебная заготовка
public IActionResult send(string id, decimal t, decimal h, int seq)
{
  if (t >= -40 || t <= 85)
  {
    data.Add(new LegacyReading(id, t, h, seq));
    return Ok();
  }
  return BadRequest();
}
```

Безопасный маршрут — POST /api/v1/telemetry. В заголовках передаются X-Device-Id, X-Key-Id, X-Timestamp, X-Nonce и X-Signature. Секретный ключ не передаётся. Сервер сам получает его из реестра устройств. Подписывается UTF-8-последовательность с разделителем LF:

```
v1
POST
/api/v1/telemetry
<deviceId>
<keyId>
<Unix timestamp>
<nonce N-format>
<lowercase SHA-256 точных байтов тела>
```

Контекст подписи включает метод и путь, поэтому подпись нельзя перенести на другую операцию. Проверяются временное окно ±120 секунд и однократность nonce. После проверки потока Request.Body.Position возвращается в 0, чтобы model binding читал те же байты тела. HMAC подтверждает целостность сообщения, но не заменяет TLS и проверку достоверности датчика.

2 Безопасная реализация и инфраструктура

```
// HmacProtocol: сравнение декодированных MAC
return CryptographicOperations.FixedTimeEquals(
  HMACSHA256.HashData(key, canonical), actual);

// Ограничение полномочий рабочего контроллера
[Authorize(Policy = "TelemetryWriter")]
[EnableRateLimiting("telemetry")]
public sealed class TelemetryController : ControllerBase
{
  // DI: ClimateDbContext, IValidator<TelemetryRequest>, IAuditSink
}

// Проверка владельца перед записью
if (request.SensorId != sensor)
  return Forbid();
var validation = await validator.ValidateAsync(request, cancellationToken);
if (!validation.IsValid)
  return BadRequest(new { code = "VALIDATION_ERROR" });

// Значения вводятся через параметры LINQ, а не SQL-конкатенацию
var query = db.Telemetry.Where(x => x.SensorId == sensor);

// Правила FluentValidation
RuleFor(x => x.TemperatureC).InclusiveBetween(-40m, 85m);
RuleFor(x => x.HumidityPercent).InclusiveBetween(0m, 100m);
```

Это выдержки из разных файлов; полная собираемая реализация находится в src/. HmacHandler выдаёт серверные claims только после проверки подписи и replay guard. Политика требует scope telemetry:write. Помещение берётся из реестра, а не из произвольного поля клиента. FluentValidation 12 поддерживает .NET 8 [1].

EF Core использует relational SQLite: в интеграционном стенде база в памяти, при самостоятельном запуске — файл climate.db. В тесте I07 получен SQL WHERE "t"."SensorId" = @__sensor_0, что подтверждает связанный параметр [2]. SQL-инъекция не объявляется исходным вектором варианта; параметризация включена как дополнительная защита.

Rate Limiting настроен на четыре запроса датчика в минуту без очереди; превышение возвращает 429 [3]. Тело ограничено 4096 байтами. Порядок middleware: routing, authentication, authorization, rate limiter. Общий лимит на входном шлюзе до аутентификации нужен отдельно.

3 Интеграционный тест закрытия уязвимости

```
[Fact]
public async Task S01_TamperedTemperatureIsRejectedBeforeWrite()
{
   using var factory = new ClimateFactory();
   using var client = factory.CreateClient();
   var payload = ClimateFactory.Payload();
   string signedBody = JsonSerializer.Serialize(payload,
      new JsonSerializerOptions(JsonSerializerDefaults.Web));
   var request = ClimateFactory.Signed(
      payload with { TemperatureC = 83.5m },
      signedBodyOverride: signedBody);
   var response = await client.SendAsync(request);
   response.StatusCode.Should().Be(HttpStatusCode.Unauthorized);
   using var scope = factory.Services.CreateScope();
   var db = scope.ServiceProvider.GetRequiredService<ClimateDbContext>();
   (await db.Telemetry.CountAsync()).Should().Be(0);
}
```

Контроль	**Полученный результат**
S01 Подмена подписанного тела	401; databaseRows=0.
S02 Чужой SensorId	403 Forbidden.
S03 Нет scope записи	403 Forbidden.
S04 Повтор nonce	401 Unauthorized.
S06 Пятый запрос	201, 201, 201, 201, 429.
S07 Превышение размера	413 Request Entity Too Large.
I07 Параметризация	В SQL есть @__sensor_0.

Результат S01 сохранён в tampering-proof.json. Тест использует настоящий HTTP-конвейер TestServer и relational-хранилище; валидный контрольный запрос I01 возвращает 201. Значение 83.5 находится внутри допустимого диапазона, поэтому отказ вызван именно нарушением подписи, а не правилом физического диапазона.

4 Реальный снимок проверки и ограничения

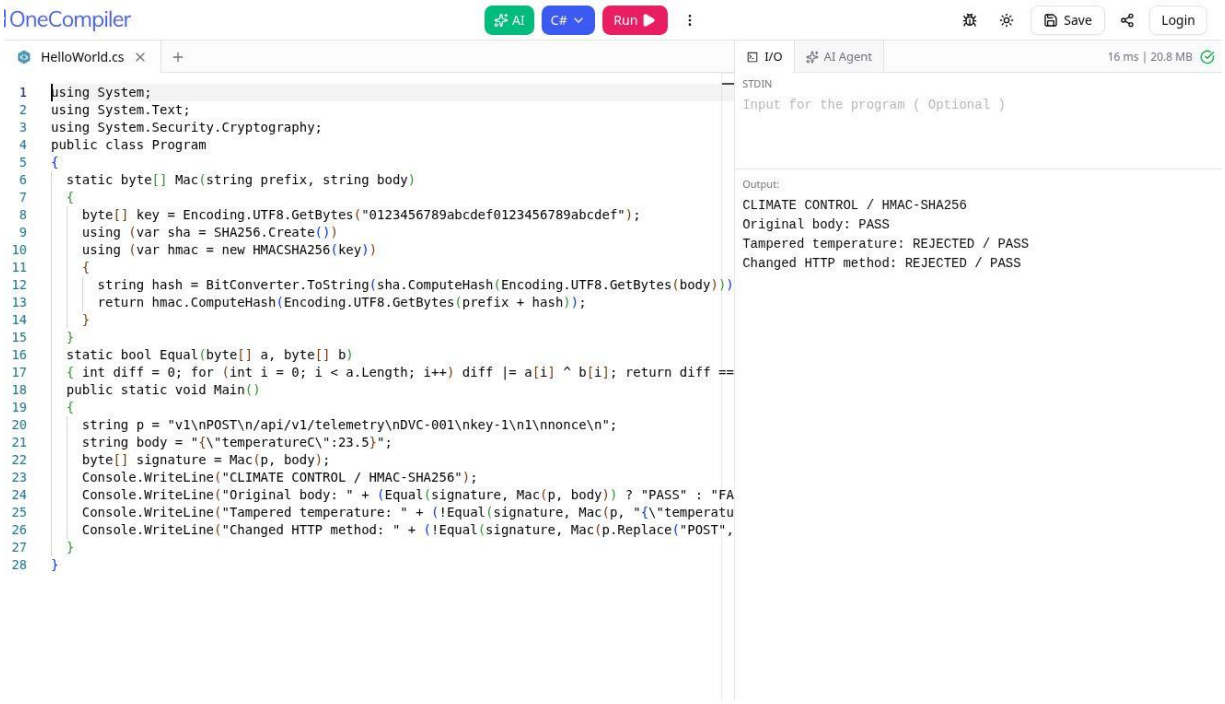


Рисунок 1 — Реальное выполнение HMAC-примера в OneCompiler

Скриншот показывает отдельную C#-демонстрацию криптографического принципа: исходное тело проходит, изменение температуры и HTTP-метода отклоняются. Использован демонстрационный ключ. Это не снимок Visual Studio и не подмена журнала xUnit; полные API-результаты находятся в integration-results.json.

Все 20 интеграционных сценариев пройдены. Для многосерверной эксплуатации необходимо вынести nonce в общий атомарный store, настроить управление и ротацию ключей, TLS и защищённый постоянный аудит. Вариант с общей памятью одного процесса не обеспечивает replay-защиту после перезапуска. Ограничения прямо отражены в проекте и не скрываются успешным тестом.

Подтверждающие материалы

Рисунок 2 дополняет материалы работы. На снимке сопоставлены три сохранённых результата стенда: принятие корректного сообщения, отказ при нарушении целостности и срабатывание ограничения частоты.

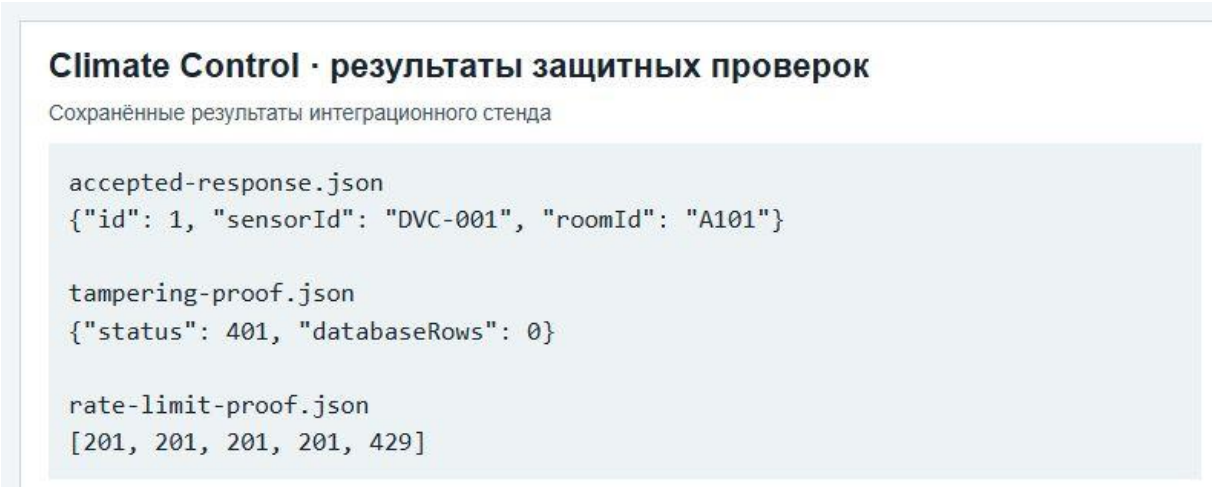


Рисунок 2 — Результаты защитных проверок стенда

Материалы работы на GitHub: открыть раздел работы

Заключение

Подмена показаний закрыта HMAC-проверкой точного тела и контекста до записи. Дополнительно внедрены FluentValidation, policy авторизации, параметризация EF Core и Rate Limiting. Интеграционный тест доказывает отказ при подмене без изменения БД.

Материалы относятся к модулю Climate Control варианта 3. Проверенный программный стенд не подключён к физическому отоплению или вентиляции кампуса. Промышленная приёмка оборудования в рамках работы не проводилась.

Список использованных источников

1. FluentValidation. Upgrade guide 12.0 and .NET 8 support. URL: https://docs.fluentvalidation.net/en/latest/upgrading-to-12.html.

2. Microsoft Learn. SQL queries in EF Core and parameterization. URL: https://learn.microsoft.com/en-us/ef/core/querying/sql-queries.

3. Microsoft Learn. Rate limiting middleware in ASP.NET Core. URL: https://learn.microsoft.com/en-us/aspnet/core/performance/rate-limit?view=aspnetcore-8.0.

4. Microsoft Learn. Integration tests in ASP.NET Core. URL: https://learn.microsoft.com/en-us/aspnet/core/test/integration-tests?view=aspnetcore-8.0.

5. Microsoft. HMACSHA256 API documentation. URL: https://learn.microsoft.com/en-us/dotnet/api/system.security.cryptography.hmacsha256.